

```
import { useState, useCallback, useRef, useEffect, useMemo } from 'react';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Progress } from '@components/ui/progress';
import { Badge } from '@components/ui/badge';
import { Label } from '@components/ui/label';
import { Separator } from '@components/ui/separator';
import {
  Sheet,
  SheetContent,
  SheetHeader,
  SheetTitle,
  SheetTrigger,
} from '@components/ui/sheet';
import {
  Popover,
  PopoverContent,
  PopoverTrigger,
} from '@components/ui/popover';
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from '@components/ui/select';
import { useToast } from '@hooks/use-toast';
import {
  Upload,
  Search,
  FileText,
  FileImage,
  Loader2,
  Trash2,
  X,
  CheckCircle2,
  XCircle,
  Clock,
  SlidersHorizontal,
  FileUp,
  FilePlus,
  AlertCircle,
} from 'lucide-react';
import { useKnowledgeBase, type Document } from '@features/ml-ai';
```

```
import { PageHeader } from '@components/navigation';
```

```
// — Types —————
```

```
interface UploadJob {  
  id: string;  
  file: File;  
  status: 'queued' | 'uploading' | 'processing' | 'completed' | 'failed';  
  jobId?: string;  
  progress: number;  
  error?: string;  
}
```

```
interface JobStatus {  
  id: string;  
  status: 'pending' | 'processing' | 'completed' | 'failed';  
  progress?: number;  
  error?: string;  
  result?: Record<string, unknown>;  
}
```

```
interface FilterState {  
  docSearch: string;  
  fileType: string;  
  status: string;  
}
```

```
interface SearchResult {  
  chunkId: string;  
  docName: string;  
  similarity: number;  
  ord: number;  
  text: string;  
}
```

```
// — Status Icon —————
```

```
function StatusIcon({ status }: { status: UploadJob['status'] }) {  
  switch (status) {  
    case 'queued':  
      return <Clock className="h-3.5 w-3.5 text-muted-foreground" />;  
    case 'uploading':  
    case 'processing':  
      return <Loader2 className="h-3.5 w-3.5 animate-spin text-blue-500" />;  
    case 'completed':  
      return <CheckCircle2 className="h-3.5 w-3.5 text-emerald-500" />;  
    case 'failed':
```

```

        return <XCircle className="h-3.5 w-3.5 text-destructive" />;
    }
}

// — Upload Panel (inside Sheet) —————
// All upload logic is preserved; only the visual presentation changes.

interface UploadPanelProps {
    uploadJobs: UploadJob[];
    setUploadJobs: React.Dispatch<React.SetStateAction<UploadJob[]>>;
    onUploadComplete: () => void;
}

function UploadPanel({ uploadJobs, setUploadJobs, onUploadComplete }: UploadPanelProps) {
    const { toast } = useToast();
    const [isDragging, setIsDragging] = useState(false);
    const fileInputRef = useRef<HTMLInputElement>(null);
    const pollingJobsRef = useRef<Map<string, AbortController>>(new Map());

    useEffect(() => {
        return () => {
            pollingJobsRef.current.forEach(c => c.abort());
            pollingJobsRef.current.clear();
        };
    }, []);

    const handleDrop = useCallback(
        (e: React.DragEvent) => {
            e.preventDefault();
            setIsDragging(false);
            const files = Array.from(e.dataTransfer.files).filter(
                f => f.type === 'application/pdf' || f.type.startsWith('image/'),
            );
            if (files.length > 0) addFiles(files);
            else
                toast({
                    title: 'Invalid files',
                    description: 'Please upload PDF or image files only.',
                    variant: 'destructive',
                });
        },
        [toast],
    );

    const handleDragOver = useCallback((e: React.DragEvent) => {
        e.preventDefault();
        setIsDragging(true);
    });

```

```
}, []));
```

```
const handleDragLeave = useCallback(() => setIsDragging(false), []);
```

```
const handleFileSelect = (e: React.ChangeEvent<HTMLInputElement>) => {  
  const files = Array.from(e.target.files ?? []);  
  if (files.length > 0) addFiles(files);  
  if (fileInputRef.current) fileInputRef.current.value = '';  
};
```

```
const addFiles = (files: File[]) => {  
  const MAX = 10 * 1024 * 1024;  
  const oversized = files.filter(f => f.size > MAX);  
  if (oversized.length > 0) {  
    toast({  
      title: 'Files too large',  
      description: `${oversized.length} file(s) exceed the 10 MB limit.`,  
      variant: 'destructive',  
    });  
  }  
  const valid = files.filter(f => f.size <= MAX);  
  if (valid.length === 0) return;  
  const newJobs: UploadJob[] = valid.map(file => ({  
    id: crypto.randomUUID().slice(0, 8),  
    file,  
    status: 'queued',  
    progress: 0,  
  }));  
  setUploadJobs(prev => [...prev, ...newJobs]);  
  newJobs.forEach(job => uploadFile(job));  
};
```

```
const uploadFile = async (job: UploadJob) => {  
  setUploadJobs(prev =>  
    prev.map(j => (j.id === job.id ? { ...j, status: 'uploading' as const, prog  
  );  
  try {  
    const formData = new FormData();  
    formData.append('file', job.file);  
    const res = await fetch('/api/kb/upload', { method: 'POST', body: formData  
    if (!res.ok) {  
      const error = await res.json();  
      throw new Error(error.message || 'Upload failed');  
    }  
    const data = await res.json();  
    setUploadJobs(prev =>  
      prev.map(j =>
```

```

        j.id === job.id
        ? { ...j, status: 'processing' as const, jobId: data.jobId, progress:
          : j,
      ),
    );
    pollJobStatus(job.id, data.jobId);
  } catch (error) {
    setUploadJobs(prev =>
      prev.map(j =>
        j.id === job.id
        ? {
            ...j,
            status: 'failed' as const,
            error: error instanceof Error ? error.message : 'Upload failed',
          }
        : j,
      ),
    );
    toast({
      title: 'Upload failed',
      description: error instanceof Error ? error.message : 'Upload failed',
      variant: 'destructive',
    });
  }
};

```

```

const pollJobStatus = async (uploadId: string, jobId: string) => {
  const ctrl = new AbortController();
  pollingJobsRef.current.set(uploadId, ctrl);
  let polling = true;
  let backoffMs = 1000;
  try {
    while (polling && !ctrl.signal.aborted) {
      try {
        const res = await fetch(`/api/kb/jobs/${jobId}`, { signal: ctrl.signal
        if (!res.ok) throw new Error('Failed to check job status');
        const status: JobStatus = await res.json();
        setUploadJobs(prev =>
          prev.map(j => {
            if (j.id !== uploadId) return j;
            if (status.status === 'completed') {
              polling = false;
              onUploadComplete();
              return { ...j, status: 'completed' as const, progress: 100 };
            }
            if (status.status === 'failed') {
              polling = false;

```

```

        return { ...j, status: 'failed' as const, error: status.error ||
    }
    return { ...j, progress: status.status === 'processing' ? 60 : 40 }
  )),
);
if (!polling) break;
await new Promise<void>((resolve, reject) => {
  const t = setTimeout(resolve, backoffMs);
  ctrl.signal.addEventListener('abort', () => {
    clearTimeout(t);
    reject(new Error('Aborted'));
  });
});
backoffMs = Math.min(backoffMs * 1.5, 5000);
} catch (error) {
  if (error instanceof Error && error.name === 'AbortError') break;
  console.error('Polling error:', error);
  await new Promise(r => setTimeout(r, 2000));
}
}
} finally {
  pollingJobsRef.current.delete(uploadId);
}
};

```

```

const removeJob = (id: string) => setUploadJobs(prev => prev.filter(j => j.id !
const clearCompleted = () =>
  setUploadJobs(prev => prev.filter(j => j.status !== 'completed' && j.status !

```

```

const activeCount = uploadJobs.filter(
  j => j.status === 'uploading' || j.status === 'processing',
).length;
const hasFinished = uploadJobs.some(j => j.status === 'completed' || j.status =

```

```

return (
  <div className="space-y-5">
    { /* Drop zone – no heavy card border, lighter feel */ }
    <div
      role="button"
      tabIndex={0}
      onKeyDown={e => {
        if (e.key === 'Enter' || e.key === ' ') {
          e.preventDefault();
          fileInputRef.current?.click();
        }
      }}
      onDrop={handleDrop}
    >

```

```

onDragOver={handleDragOver}
onDragLeave={handleDragLeave}
onClick={() => fileInputRef.current?.click()}
className=[
  'border border-dashed rounded-xl p-8 text-center cursor-pointer',
  'transition-all duration-200',
  isDragging
    ? 'border-primary bg-primary/5 scale-[1.01]'
    : 'border-border hover:border-primary/40 hover:bg-muted/20',
].join(' ')
data-testid="dropzone-upload"
>
<div className="flex flex-col items-center gap-3">
  <div
    className=[
      'p-3 rounded-full transition-colors',
      isDragging ? 'bg-primary/10' : 'bg-muted',
    ].join(' ')
  >
    <FileUp
      className=[
        'h-5 w-5 transition-colors',
        isDragging ? 'text-primary' : 'text-muted-foreground',
      ].join(' ')
    />
  </div>
  <div>
    <p className="text-sm font-medium">Drop files here</p>
    <p className="text-xs text-muted-foreground mt-1">
      or click to browse · PDF, PNG, JPEG · Max 10 MB
    </p>
  </div>
</div>
<input
  ref={fileInputRef}
  type="file"
  multiple
  accept=".pdf,image/png,image/jpeg"
  onChange={handleFileSelect}
  className="hidden"
  data-testid="input-file-multiple"
/>
</div>

{/* Queue — compact rows, no per-item cards */}
{uploadJobs.length > 0 && (
  <div>

```

```

<div className="flex items-center justify-between mb-2">
  <p className="text-xs font-medium text-muted-foreground">
    {activeCount > 0
      ? `${activeCount} processing...`
      : `${uploadJobs.length} file${uploadJobs.length !== 1 ? 's' : ''}
    }
  </p>
  {hasFinished && activeCount === 0 && (
    <Button
      variant="ghost"
      size="sm"
      className="h-6 px-2 text-xs"
      onClick={clearCompleted}
      data-testid="button-clear-completed"
    >
      Clear all
    </Button>
  )}
</div>

```

```

<div className="space-y-1" data-testid="upload-queue">
  {uploadJobs.map(job => (
    <div
      key={job.id}
      className="flex items-center gap-2.5 px-3 py-2 rounded-lg bg-mute"
      data-testid={`upload-job-${job.id}`}
    >
      <StatusIcon status={job.status} />

      <div className="flex-1 min-w-0">
        <p
          className="text-xs font-medium truncate leading-tight"
          data-testid={`text-filename-${job.id}`}
        >
          {job.file.name}
        </p>
        {job.error && (
          <p
            className="text-xs text-destructive leading-tight mt-0.5"
            data-testid={`text-error-${job.id}`}
          >
            {job.error}
          </p>
        )}
      </div>
      {(job.status === 'uploading' || job.status === 'processing') &&
        <Progress
          value={job.progress}
          className="h-0.5 mt-1.5"
        >

```



```

        data-testid={`progress-${job.id}`}
      />
    ))
  </div>

  <span
    className="text-xs text-muted-foreground capitalize shrink-0"
    data-testid={`text-status-${job.id}`}
  >
    {job.status}
  </span>

  {(job.status === 'completed' || job.status === 'failed') && (
    <button
      onClick={() => removeJob(job.id)}
      className="text-muted-foreground hover:text-foreground transi
      aria-label="Remove"
      data-testid={`button-remove-${job.id}`}
    >
      <X className="h-3.5 w-3.5" />
    </button>
  )}
</div>
  )})
</div>
</div>
  )}
</div>
);
}

```

// — Filter Popover —————

// Hidden by default. Active filters shown as a count badge on the trigger.

```

interface FilterPopoverProps {
  filters: FilterState;
  onChange: (key: keyof FilterState, value: string) => void;
  onClear: () => void;
  activeCount: number;
}

function FilterPopover({ filters, onChange, onClear, activeCount }: FilterPopover
  return (
    <Popover>
      <PopoverTrigger asChild>
        <Button
          variant="ghost"

```

```

        size="sm"
        className="h-7 gap-1.5 text-muted-foreground hover:text-foreground px-2
    >
    <SlidersHorizontal className="h-3.5 w-3.5" />
    <span className="text-xs">Filter</span>
    {activeCount > 0 && (
        <Badge
            variant="secondary"
            className="h-4 min-w-4 px-1 text-xs rounded-full"
        >
            {activeCount}
        </Badge>
    )}
    </Button>
</PopoverTrigger>
<PopoverContent align="end" className="w-60 p-4 space-y-4">
    <div className="space-y-1.5">
        <Label className="text-xs font-medium text-muted-foreground">Search nam
    <Input
        placeholder="Filter by name..."
        value={filters.docSearch}
        onChange={e => onChange('docSearch', e.target.value)}
        className="h-8 text-sm"
    />
    </div>

    <div className="space-y-1.5">
        <Label className="text-xs font-medium text-muted-foreground">File type<
        <Select value={filters.fileType} onValueChange={v => onChange('fileType
            <SelectTrigger className="h-8 text-sm">
                <SelectValue placeholder="All types" />
            </SelectTrigger>
            <SelectContent>
                <SelectItem value="all">All types</SelectItem>
                <SelectItem value="pdf">PDF</SelectItem>
                <SelectItem value="png">PNG</SelectItem>
                <SelectItem value="jpeg">JPEG</SelectItem>
            </SelectContent>
        </Select>
    </div>

    <div className="space-y-1.5">
        <Label className="text-xs font-medium text-muted-foreground">Status</La
        <Select value={filters.status} onValueChange={v => onChange('status', v
            <SelectTrigger className="h-8 text-sm">
                <SelectValue placeholder="All statuses" />
            </SelectTrigger>

```

```

        <SelectContent>
          <SelectItem value="all">All statuses</SelectItem>
          <SelectItem value="ready">Ready</SelectItem>
          <SelectItem value="processing">Processing</SelectItem>
          <SelectItem value="failed">Failed</SelectItem>
        </SelectContent>
      </Select>
    </div>

    {activeCount > 0 && (
      <>
        <Separator />
        <Button
          variant="ghost"
          size="sm"
          className="w-full h-7 text-xs"
          onClick={onClear}
        >
          Clear filters
        </Button>
      </>
    )}
  </PopoverContent>
</Popover>
);
}

// — Document Row —————
// No per-row cards or borders – hover state only. Delete reveals on hover.

interface DocumentRowProps {
  doc: Document;
  onDelete: (id: string, name: string) => void;
  isDeleting: boolean;
}

function DocumentRow({ doc, onDelete, isDeleting }: DocumentRowProps) {
  const isImage = ['png', 'jpeg', 'jpg', 'webp'].includes(doc.fileType.toLowerCase)

  return (
    <div
      className="group flex items-center gap-3 px-3 py-2.5 rounded-lg hover:bg-mu
      data-testid={`document-${doc.id}`}
    >
      {/* File type icon */}
      <div className="shrink-0 p-1.5 rounded-md bg-muted text-muted-foreground">
        {isImage ? (

```

```

        <FileImage className="h-3.5 w-3.5" />
      ) : (
        <FileText className="h-3.5 w-3.5" />
      )}
    </div>

    { /* Name + meta */ }
    <div className="flex-1 min-w-0">
      <p
        className="text-sm font-medium truncate leading-tight"
        data-testid={`text-name-${doc.id}`}
      >
        {doc.name}
      </p>
      <p className="text-xs text-muted-foreground mt-0.5">
        {doc.fileType.toUpperCase()} · {doc.numChunks} chunks · {' '}
        {new Date(doc.createdAt).toLocaleDateString()}
      </p>
    </div>

    { /* Delete — reveals on row hover */ }
    <button
      onClick={() => onDelete(doc.id, doc.name)}
      disabled={isDeleting}
      className={[
        'opacity-0 group-hover:opacity-100 transition-all',
        'text-muted-foreground hover:text-destructive',
        'disabled:pointer-events-none disabled:opacity-30',
      ].join(' ')}
      aria-label={`Delete ${doc.name}`}
      data-testid={`button-delete-${doc.id}`}
    >
      <Trash2 className="h-3.5 w-3.5" />
    </button>
  </div>
);
}

// — Search Result Item —————
// Divider-separated entries, no per-result cards.

function SearchResultItem({ result }: { result: SearchResult }) {
  return (
    <div className="py-4 border-b border-border/50 last:border-0">
      <div className="flex items-baseline gap-2 mb-1.5">
        <span
          className="text-xs font-semibold text-foreground"

```

```

        data-testid={`text-doc-name-${result.chunkId}`}
      >
        {result.docName}
      </span>
      <span className="text-xs text-muted-foreground">
        {(result.similarity * 100).toFixed(1)}% match · Chunk {result.ord + 1}
      </span>
    </div>
    <p
      className="text-sm text-muted-foreground leading-relaxed line-clamp-4"
      data-testid={`text-chunk-${result.chunkId}`}
    >
      {result.text}
    </p>
  </div>
);
}

```

// — Main Page —

```

export default function KnowledgeBasePage() {
  const {
    stats,
    documentsData,
    documentsLoading,
    searchQuery,
    setSearchQuery,
    searchData,
    searching,
    handleUploadComplete,
    handleDelete,
    deleteMutation,
  } = useKnowledgeBase();

  // Lifted upload state so the header button can reflect active-upload count
  const [uploadJobs, setUploadJobs] = useState<UploadJob[]>([]);
  const activeUploads = uploadJobs.filter(
    j => j.status === 'uploading' || j.status === 'processing',
  ).length;

  // Filter state - starts fully collapsed (all = "all")
  const [filters, setFilters] = useState<FilterState>({
    docSearch: '',
    fileType: 'all',
    status: 'all',
  });
}

```

```

const updateFilter = useCallback((key: keyof FilterState, value: string) => {
  setFilters(prev => ({ ...prev, [key]: value }));
}, []);

const clearFilters = useCallback(() => {
  setFilters({ docSearch: '', fileType: 'all', status: 'all' });
}, []);

const activeFilterCount = [
  filters.docSearch !== '',
  filters.fileType !== 'all',
  filters.status !== 'all',
].filter(Boolean).length;

// Search state helpers
const isSearchMode = searchQuery.length > 0;
const searchReady = searchQuery.length >= 3;

// Filtered document list
const filteredDocuments = useMemo(() => {
  if (!documentsData?.documents) return [];
  return documentsData.documents.filter((doc: Document) => {
    const matchesSearch =
      !filters.docSearch ||
      doc.name.toLowerCase().includes(filters.docSearch.toLowerCase());
    const matchesType = filters.fileType === 'all' || doc.fileType === filters.fileType;
    const matchesStatus = filters.status === 'all' || doc.status === filters.status;
    return matchesSearch && matchesType && matchesStatus;
  });
}, [documentsData?.documents, filters]);

const totalDocs = documentsData?.documents?.length ?? 0;

return (
  <div className="min-h-screen" data-testid="page-knowledge-base">
    <PageHeader title="Knowledge Base" />

    {/* — Narrow centered column — calmer, focused ————— */}
    <div className="container mx-auto px-6 py-8 max-w-3xl space-y-8">

      {/* — Top bar: stats (secondary) + Upload action ————— */}
      <div className="flex items-center justify-between">
        {/* Stats — minimal, muted, not competing with content */}
        <div className="flex items-center gap-2 text-xs text-muted-foreground">
          {stats ? (
            <>
              <span data-testid="text-total-documents">{stats.totalDocuments} d

```

```

        <span className="text-muted-foreground/30">.</span>
        <span data-testid="text-total-chunks">{stats.totalChunks} chunks<
    </>
    ) : (
        <span className="opacity-0 select-none"></span> // reserves height
    )}
</div>

{/* Upload - Sheet keeps it out of the main flow entirely */}
<Sheet>
    <SheetTrigger asChild>
        <Button variant="outline" size="sm" className="gap-2 h-8 text-xs">
            {activeUploads > 0 ? (
                <Loader2 className="h-3.5 w-3.5 animate-spin" />
            ) : (
                <Upload className="h-3.5 w-3.5" />
            )}
            Upload
            {activeUploads > 0 && (
                <Badge
                    variant="secondary"
                    className="h-4 min-w-4 px-1 text-xs rounded-full ml-0.5"
                >
                    {activeUploads}
                </Badge>
            )}
        </Button>
    </SheetTrigger>

    <SheetContent className="w-full sm:max-w-md overflow-y-auto">
        <SheetHeader className="mb-6">
            <SheetTitle>Upload Documents</SheetTitle>
            <p className="text-sm text-muted-foreground">
                Add PDFs or images to your knowledge base. They'll be processed
                automatically.
            </p>
        </SheetHeader>
        <UploadPanel
            uploadJobs={uploadJobs}
            setUploadJobs={setUploadJobs}
            onUploadComplete={handleUploadComplete}
        />
    </SheetContent>
</Sheet>
</div>

{/* — Search — the page's primary action ————— */}

```

```

<div className="space-y-1.5">
  <div className="relative">
    <Search className="absolute left-3.5 top-1/2 -translate-y-1/2 h-4 w-4
    {searching && (
      <Loader2 className="absolute right-3.5 top-1/2 -translate-y-1/2 h-4
    )}
    <Input
      placeholder="Ask a question or search for information..."
      value={searchQuery}
      onChange={e => setSearchQuery(e.target.value)}
      className="pl-10 pr-10 h-11 text-sm bg-background shadow-sm"
      data-testid="input-search-query"
    />
  </div>

  {/* Inline hint — only visible while typing, before 3 chars */}
  {isSearchMode && !searchReady && (
    <p className="text-xs text-muted-foreground flex items-center gap-1.5"
    <AlertCircle className="h-3 w-3" />
    Type at least 3 characters to search
  </p>
  )}
</div>

{/* — Search results — shown when query is ready ————— *
{searchReady && (
  <div data-testid="search-results">
    {searchData?.results.length > 0 ? (
      <div>
        <p className="text-xs text-muted-foreground mb-3">
          {searchData.results.length} result{searchData.results.length !=
        </p>
        {searchData.results.map((result: SearchResult) => (
          <SearchResultItem key={result.chunkId} result={result} />
        )}}
      </div>
    ) : searchData ? (
      <div className="text-center py-12">
        <p className="text-sm text-muted-foreground">
          No results found for &ldquo;{searchQuery}&rdquo;
        </p>
        <p className="text-xs text-muted-foreground mt-1">
          Try different keywords or upload more documents
        </p>
      </div>
    ) : null}
  </div>

```



```

    })

    { /* — Documents list — hidden while search is active ————— *
    { !isSearchMode && (
      <div className="space-y-2">
        { /* Section header: count + filter trigger */
        <div className="flex items-center justify-between h-8">
          <span className="text-sm font-medium text-muted-foreground">
            { !documentsLoading && totalDocs > 0
              ? `${filteredDocuments.length} of ${totalDocs}`
              : 'Documents'
            }
          </span>
          { totalDocs > 0 && (
            <FilterPopover
              filters={filters}
              onChange={updateFilter}
              onClear={clearFilters}
              activeCount={activeFilterCount}
            />
          ) }
        </div>

        { /* Loading */
        { documentsLoading && (
          <div className="flex justify-center py-12">
            <Loader2 className="h-5 w-5 animate-spin text-muted-foreground" />
          </div>
        ) }

        { /* Document rows — no borders, hover-only background */
        { !documentsLoading && filteredDocuments.length > 0 && (
          <div className="-mx-3" data-testid="documents-list">
            { filteredDocuments.map((doc: Document) => (
              <DocumentRow
                key={doc.id}
                doc={doc}
                onDelete={handleDelete}
                isDeleting={deleteMutation.isPending}
              />
            ) ) }
          </div>
        ) }

        { /* Empty — no documents uploaded yet */
        { !documentsLoading && totalDocs === 0 && (
          <div className="text-center py-16 space-y-2">
            <FilePlus className="h-8 w-8 text-muted-foreground/20 mx-auto" />

```

```

        <p className="text-sm text-muted-foreground">No documents yet</p>
        <p className="text-xs text-muted-foreground">
            Use the Upload button to add your first document
        </p>
    </div>
  )}

  { /* Empty - filters exclude all results */ }
  { !documentsLoading && totalDocs > 0 && filteredDocuments.length === 0
    <div className="text-center py-10">
      <p className="text-sm text-muted-foreground">No documents match y
      <Button
        variant="link"
        size="sm"
        className="text-xs mt-1 h-auto p-0"
        onClick={clearFilters}
      >
        Clear filters
      </Button>
    </div>
  )}
</div>
)}
</div>
</div>
);
}

```